

ON BUILDING

SYSTEMS

Single Agent Loops and Systems



#ASLI ENGINEERING

BY

ARPIT BHAYANI

Agenda

1. Observe-Think-Act 5 mins
 2. Ralph Loop 10 mins
 3. ReAct Loop 5 mins
 4. Plan-and-Execute Loop 5 mins
-
5. File System as Context 7 min
 6. Long Running Agents: Checkpoint and Resume 5 min
 7. Human-In-The-loop 7 min
-
- 10 mins QnA

Prototypes: 6

System: Paper to Code System

Appendix:

- Reflexion
- Checkpoint / Resume for Workflows and Toolings



Arpit Bhayani
@arpit_bhayani

Fuck! AI agent is just an expensive while loop.

Last edited 7:27 PM · Jun 7, 2026 · 287K Views

250

195

4.2K

232



Agent Fundamentals

Agents → autonomously reasons, plans, and take actions to achieve specific goals.

utilizes tools, memory, and **multistep reasoning**

Observe-Think-Act loop

1. observe the environment

2. reason about what to do next

3. act → get feedback

whatever agent can read: prompt, tool output, file content, errors, etc

Repeat until

the task is complete

Feedback loop

→ whatever agent can write: tool call, file edit, message, etc

* model in single pass → hallucinate → wrong answer

observe → steps → errors/mistakes gets corrected at every step

Practical use:

Web Scraping on Dynamic Pages having Login, Pagination, etc

QA Testing - Test, observe errors, fix, repeat

Debugging a code - run, observe, fix, repeat

Form Filling with dynamic fields - fill, observe, fill, repeat

see the pattern now?

Prototype: Patch Bugs in Python code

shell-level orchestration pattern

wraps any CLI based coding agent in while loop

LLM thinks, it is done! esp coding agent

Ralph loop

Key problem: Decision to stop the loop is probabilistic

1. agent exits when it thinks it is done, not when the tests pass
2. some agents require manual re-prompting to keep moving
3. engineer becomes the loop controller

Response: Plan-Execute, ReAct, Observe Reason Act, etc

Ralph treats problem as control flow problem, and not reasoning.

while :; do [completion criteria] * File System and Git History *

cat prompt.md | claude --continue

Loop memory is not inside context window
but memory is the repository itself.

done

Full reset per iter!

High simplicity

Treats File System as source of truth

Each iteration gets a fresh allocation of working space

spec.md

Prototype: Code Patcher

Multiple bugs

```
# Task
Add unit tests for all public methods in 'src/parser.py'.

## Completion criteria
- Test file exists at 'tests/test_parser.py'
- All tests pass with 'pytest tests/test_parser.py'
- Coverage of 'src/parser.py' is above 80% as reported by 'pytest --cov=src/parser'
- Output the exact string 'COMPLETE' when all criteria are satisfied

## Constraints
- Do not modify 'src/parser.py'
- Do not add any new dependencies beyond 'pytest' and 'pytest-cov'
- Commit progress incrementally with descriptive messages
```

Similar to Observe Think Act

but thought is explicit and written.

ReAct - Reasoning + Acting loop

Thought is a first class citizen.

Each iteration produces a thought and action
and waits for an observation

shapes the next action

we are making agent
reason out loud and
surface errors early.

when reasoning is wrong we can see it!

* infinite loop possible, hence max-steps

↳ essential in production

avoid burning \$4 to fetch URL which keeps giving 403

Prototype: Search for population of India,
then calculate 0.1% of it

Thought is
precise

Tools: Search, calculator, and Summarizer

* ReAct can lose sight of original goal across many steps

Practical use:

1. Cx Bot that reasons through the complaint before deciding
2. SQL query builder: thinks about schema and writes query
3. Travel Planner: checks flight, reasons about layover time
4. Medical Triage: reasons about symptoms
5. Email Agent: reasons about email, sender, urgency, etc before deciding.

Then use OTA loop
to make sure it
is correct

1. Research Report writer - outline & research one by one
2. software feature builder - plan, design, implement, test
3. Trip itinerary planner - day by day

Plan-and-Execute loop 4. competitive analysis - info gather, point by point analysis

Given a task

1. planner - generates a full task breakdown upfront
2. executor - works through each subtask
3. replanner - adjust when execution diverges from the plan

Parallel

Trade-off: Brittleness

↳ subtask that planner assumed would succeed, actually failed

could not complete

This may trigger a full replan

For open ended tasks, ReAct is better

↳ flexible in execution

In production: it is always a hybrid → we see this in Claude Code

↳ high level outline is planned

↳ then ReAct handles execution at each step

Prototype: Calculate total procurement cost for a junior developer setup:

1x MacBook M1 Pro

2x 27-inch 4k monitors

1x Standing Desk

1x Ergonomic Chair

1x Membrane keyboard

1x wireless mouse

Provide

- itemized breakdown

- grand bill

File System as Context

When coding agent starts

it should never start fresh - reading already read files
rediscovering some conclusions again

"I have read auth/service.go, auth/middleware.go, and user/model.go"

before reading a file ↗

↳ agent checks the list

prevents overrides

↓ *

before writing, agent confirms that its cached version is still the current
before reading, agent checks if already in context.... skip!

This also helps when you are 'indexing' files

↳ eg: inverted index of text file

* no need to update index when it hasn't changed

* Workflow checkpoint → Temporal
→ Agno, DBOS, etc

Checkpoint and Resume

short agents can retry cheaply
but not longer ones

Running AI applications in production
is much more than just making
some LLM and tool calls!

Real agentic workflows span HOURS!

* crash, rate limits, etc are inevitable
n/w timeouts, OOM, pod eviction/rotation, HITL power,
context window overflow, etc

Save the agent state!

↳ not that simple though

Checkpointing and Resume

→ Database, Flat file

Every successful step → take full dump as checkpoint → Task specific

history → every prompt sent, every response received

metadata → model config, task definitions, timestamps

contextual integrity → exact 'memory' of agent

* claude session json file

Restore: load entire conversation in-mem

reconstruct 'context' (file system, git workdir, etc)

pass this entire to LLM for next iter.

Prototype: Checkpoint and Resume

Human In the loop

Default: Full autonomous / Sometimes stakes are high: HITL

1. Tool level confirmation - decorate a tool call & confirm right before tool exec.
2. Confidence based - usecase specific (support ticket)
3. Workflow level checkpoints - pause at major step of a workflow

Ways to implement

1. Synchronous blocking - input(), dialog on UI (Claude)
 2. Webhook + callback - agent polls a URL until approved
 3. Queue based - approval queues
4. Tool call interception - when LLM thinks it needs human input, it does tool call

Practical use:

Approval Gates: confirm before critical / sensitive action → email, submit form, etc

Ambiguity Resolution: choosing b/w conflicting, are you sure Q3 2025?

Risk checkpoint: Deletions

Exception & Anomalies: amount exceeds threshold, discounts, etc

Quality Review: PR, review generated test suite, social post review

Compliance & audit: Human sign off required before sharing the report

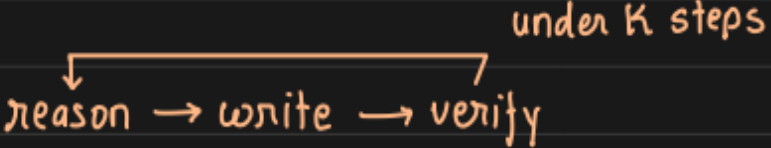
Prototype: HITL as Tool use

Paper To Code System

Command line agent that reads a single research paper
and produces working Python code

* no server, no database,

Dominant design pressure: CORRECTNESS

What should agent do? 
reason → write → verify

solutioning...

1. Functional and Non-functional requirements
2. Tools needed
3. High level workflow
4. ReAct loop - why not Ralph and Plan-and-execute
5. Code execution
6. Prompts
7. any other step we are missing?

Brainstorm

paper → code

↳ ocr tool / pdf extract / ... ↗ ext image

↳ read write file ...

↳ ~~data~~ / python -dtk-exec

↳ alebsearch → use this tool for text / concept that you are unconv. of noting etc

↳ get_section ("results")
 'approach'
 'algo'

Exactly match

↳ headline should be seen
 "reproduce the results"

Functional Requirements

1. ingest, extract, write, test, iterate
2. agent reasons about central contribution of the paper
3. code execution via subprocess
4. tests the code, and repeat

Function calls for agents
"decomposable"

Non Functional Requirements

1. Step budget
2. Code execution Isolation
3. Correctness

Tools Required

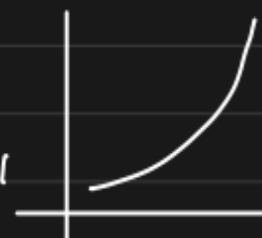
1. Extract section
2. write code
3. run code
4. finish

"Capacity Estimations"

Cumulative tokens consumed till k^{th} step

2k-4k token per step, by 10^{th} step 20k-40k token in each call

Using this, estimate the cost/run



High level flow

```
user invokes main.py
-> ingest.py reads paper -> paper_text (str)
-> react_loop.run(AgentState)
    -> Anthropic API (system prompt + paper_text + messages)
    <- response (tool_use blocks)
    -> tools.dispatch_tool(name, args, paper_text)
    <- (result_str, is_done, code)
    -> append tool_result to messages
    -> loop
<- final_code (str)
-> main.py writes final_code to output_path
```

Read section

→ write and modify code

test code

...

ReAct loop

Ralph suits when environment / state changes often

Plan and execute works well when we have predictable subtasks

Interleaving reasoning and acting → "improving output/code"

↳ as we keep discovering new info

Code Execution

- subprocess and timeout
- tool call determines next improvements and when to run
- if error, agent fixes in scratchpad

Prompts

```
You are an expert software engineer that implements algorithms and systems from \
research papers.
```

```
## Your task
```

```
Read the paper below and produce working, well-commented Python code that implements the \
core contribution of the paper. The code must be saved to: {output_path}
```

```
## ReAct loop instructions
```

```
You operate in a Reason-Act loop. At every step you MUST:
```

1. Write a short THOUGHT explaining what you will do and why.
2. Call exactly ONE tool to act on that thought.

```
Available tools:
```

- 'extract_section' -- pull a named section from the paper for closer inspection
- 'write_code' -- write a complete Python code block to a scratch buffer
- 'run_code' -- execute the current scratch buffer; returns stdout/stderr
- 'finish' -- save the final code to disk and terminate the loop

```
Rules:
```

- Always think before acting.
- After 'run_code', inspect the output. If there are errors, fix them with 'write_code' \
then 'run_code' again.
- Call 'finish' only when the code is correct (run_code showed no fatal errors).
- Do NOT call 'finish' before running the code at least once.
- Keep the implementation focused on the paper's core algorithm. Do not pad with unrelated code.

```
## Paper text
```

```
---
{paper_text[:60_000]}
---
```

→ given a section name,
give me the section

"not all sections are relevant"

```
# Seed the conversation with the initial user request
```

```
state.messages.append(
    {
        "role": "user",
        "content": (
            "Implement the core algorithm from the paper as working Python code. "
            "Begin by reasoning about the paper's central contribution, then act step by step."
        ),
    }
)
```

→ seeding the conversation

```
state.messages.append(
    {
        "role": "user",
        "content": (
            "You haven't called 'finish' yet. "
            "If the code is ready, call 'finish'. "
            "Otherwise continue debugging."
        ),
    }
)
```

→ on end turn so that
scratch pad is written to disk

Appendix

Reflexion

ReAct agent observe tool output and adjust

They do not reflect why series of action failed!

* if approach is fundamentally flawed, ReAct will exhaust step limit

→ adds a layer above the loop

and agent generates verbal reflection

kind of self improving

and stores it in the memory buffer.

↪ software

weight-free learning → injected in next iteration → Generate Rich Signal

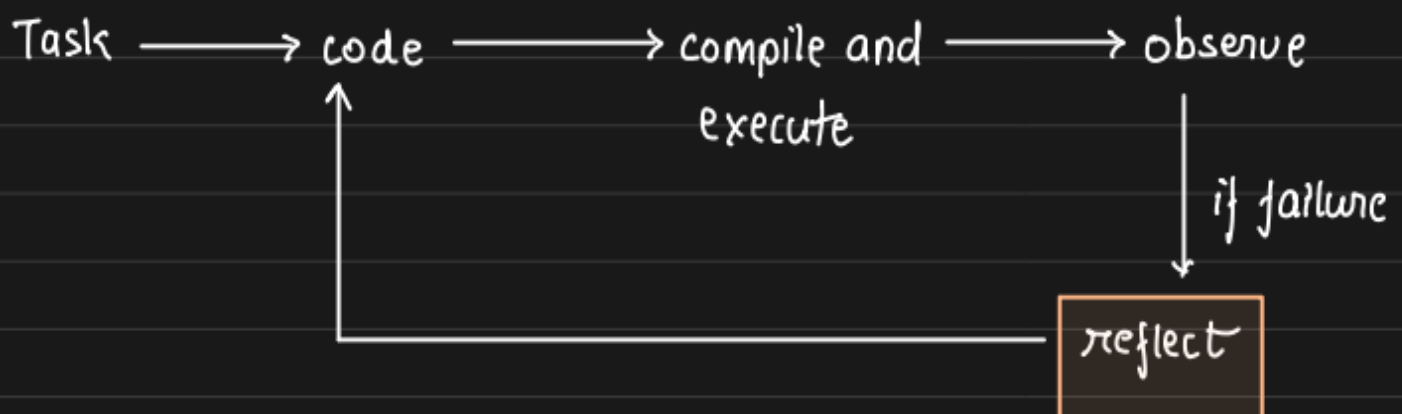
max impact on sequential decision making, coding, and reasoning tasks.

* max retry should always be there

Prototype: Implement merge intervals using Reflexion

↪ 2 hidden req

* First attempt often misses 2/3 edge cases



Checkpoint in workflows & toolings

state: execution state of the workflow + conversation



1. complete graph state at current node
2. output of all completed nodes
3. input and output at each node
4. output of tool calls, etc

} serialized
msg history

Checkpoint after every
single execution step!

* storage is cheap

retries are expensive

* langGraph gives you checkpointing out of the box (PG, SQLite, Memory)

* Temporal - durable execution as a primitive

↓ Handles state persistence, retry logic, and fault tolerance transparently

Is it safe to retry? - Idempotency keys

Temporal has activity ID

ex. Send notif, phone call, etc